SCCAD 3nm PDK

User Manual

Cadence Innovus Edition

Version	v1.0
Release Date	TBD
Organization	SCCAD Lab, University of Southern California
Tool	Cadence Innovus
Contact	sw.jung@gatech (Sungwoo Jung)

This manual provides comprehensive guidance for users and developers working with the **SCCAD 3nm Process Design Kit (PDK)**. It covers PDK installation from GitHub, environment configuration, and step-by-step instructions for running Place-and-Route (PnR) using **Cadence Innovus**. Additionally, it provides guidance on how to modify key PDK components — including SPICE model cards and interconnect models (ITF/RC).

Table of Contents

1 Introduction

1.1 Overview of SCCAD 3nm PDK

The **SCCAD 3nm Process Design Kit (PDK)** is developed and maintained by the SCCAD Laboratory to support academic and research-oriented digital IC design flows at the 3 nm technology node. The PDK provides a complete set of design enablement files — including technology files, standard-cell libraries, SPICE models, and interconnect models — that allow designers to perform synthesis, place-and-route (PnR), and sign-off verification using industry-standard EDA tools.

Several open-source 3 nm PDKs are available to the research community. The table below positions SCCAD-3nm alongside two other publicly available 3 nm PDKs — FreePDK3 (NCSU) and GT3 (Gatech) — to highlight the distinguishing capabilities of this release.

Feature	FreePDK3 (NCSU)	GT3 (Gatech)	SCCAD-3nm
Tech Node	3nm	3nm	3nm
Device Type	GAAFET	GAAFET	GAAFET
Cell Heights	5.5T	6T	**6T, 5T**
Power Rail (PR)	Front-side (FSPR)	Buried (BPR)	**FSPR, BPR**
Support Back-side Metal	No	No	**Yes**

As shown in the table, all three PDKs target the 3 nm node and adopt Gate-All-Around FET (GAAFET) device architectures. Within this shared foundation, the SCCAD-3nm PDK offers several capabilities that extend the design space available to researchers.

- **Dual cell-height support (6T and 5T).** Providing both 6-track and 5-track standard-cell libraries gives designers flexibility to trade off area density against routability, enabling exploration of different optimization targets within a single PDK.
- **Both Front-side and Buried Power Rail (FSPR & BPR).** Supporting both power delivery architectures allows direct comparison of conventional front-side routing against buried power rail approaches.
- **Back-side metal support.** SCCAD-3nm includes back-side metal interconnect, enabling exploration of back-side power delivery network (BS-PDN) and signal routing techniques central to next-generation 3D integration research.

Together, these features make SCCAD-3nm well-suited for research spanning standard-cell library characterization, power delivery network analysis, back-side interconnect exploration, and full back-end PnR flows at an advanced technology node.

1.2 Key Features and Technology Highlights

Technology Node: 3 nm GAAFET

The SCCAD-3nm PDK is built on a **3 nm Gate-All-Around FET (GAAFET)** process. Unlike FinFET, where the gate wraps around three sides of a fin, GAAFET surrounds the channel on all four sides using stacked nanosheet or nanowire structures. This architecture provides superior electrostatic control, reduced short-channel effects, and improved drive current at scaled supply voltages. The fabrication process sequence is illustrated in **Figure 1**.

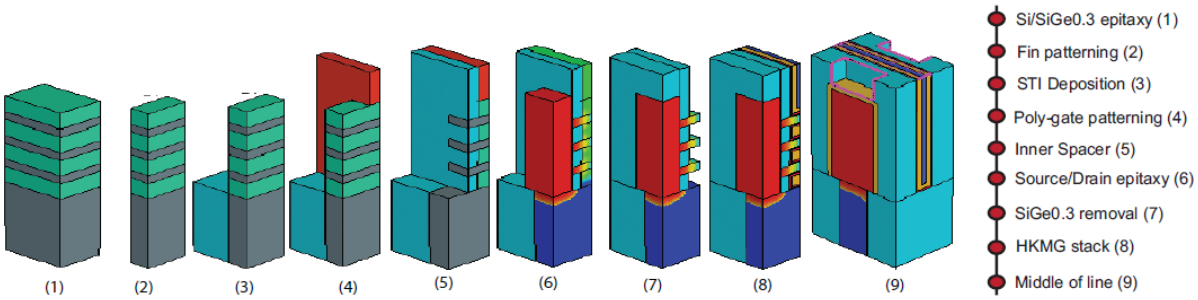

Figure 1. TCAD device structure of the SCCAD 3nm GAAFET.

Table 1 summarizes the key device physical parameters of the SCCAD-3nm NSFET device in comparison with a representative 5 nm FinFET process.

Table 1. Comparison of device physical parameters of FinFET and NSFET device.

Physical Parameters (nm)	TSMC 5nm	NSFET 3nm
CPP	48	42
Fin/NS pitch	25	NA
Gate length	15	12
Spacer length	6	5
# Fin/NS	2	3
Fin/NS thickness	6	5
Fin/NS width	NA	30, 20
Fin/NS stack height	55	55

Standard-Cell Library

The SCCAD-3nm PDK includes two standard-cell libraries. The **6-Track (6T) library** uses a Front-side Power Rail (FSPR) architecture, and the **5-Track (5T) library** adopts a Buried Power Rail (BPR) architecture. The complete cell set comprises **57 standard cells**. Table 2 lists the full cell set.

STD Cells	(Input) x (#parallel Trs)
INV	x1, x2, x3, x4, x5, x6, x7, x8, x10, x12, x14, x16
BUF	x1, x2, x3, x4, x5, x6, x7, x8, x10
NAND, NOR	2x1, 3x1, 4x1, 2x2, 3x2
AND, OR	2x1, 3x1, 4x1
XOR, XNOR	2x1, 3x1
AOI, OAI	(21, 22, 221, 222, 311, 31) x1
MUX	2x1
D Flip-Flop	Hx1 (async.), HQNx1 (sync.)
DHL (latch)	x1
Total	57

Figure 2 shows the INVx1 cell layout for each library variant.

6-Track (6T) / FSPR: Power rails on **M1**, cell height **144 nm**, nanosheet width **30 nm**.

5-Track (5T) / BPR: Power rails on buried **MBPR** layer via VBPR vias (12 nm × 18 nm), MBPR width **25 nm**, cell height **120 nm**.

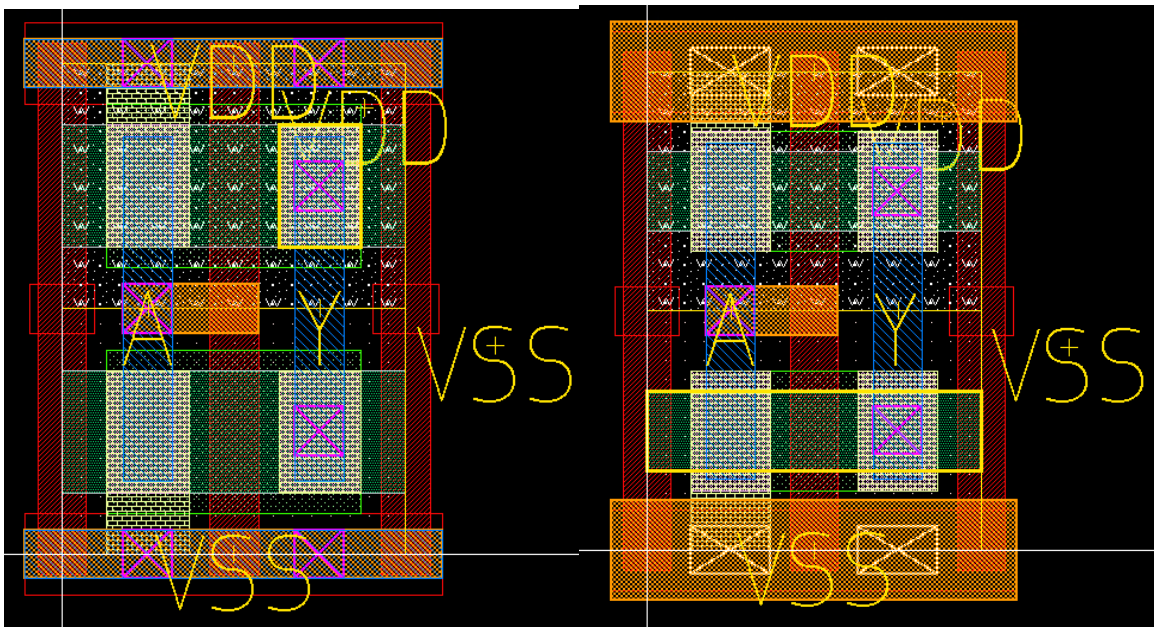

Figure 2a. INVx1 layout — 6T / FSPR library. *Figure 2b. INVx1 layout — 5T / BPR library.*

Back-end-of-Line (BEOL) Technology

The SCCAD-3nm PDK features a comprehensive BEOL stack from MB2 through M9 and via layers VB1 through V8. Table 3 summarizes physical parameters.

	Metal	W (nm)	Resistance (Ω)
Back-side	MB2-MB1	61	11
BPR cell layer	MBPR	25	65
Front-Side	M0	20	523
	M1-M3	12	347
	M4, M5	18	101
	M6, M7	24	44
	VIA	**W × L (nm²)**	**Resistance (Ω)**
Back-Side	VB1	40 × 40	4.0
BPR cell layer	VBPR	20 × 12	56
BSP cell layer	µ-TSV	60 × 60	5
	BSC	20 × 20	74.9
Front-Side	V0-V3	12 × 12	63.5
	V4, V5	18 × 18	19.38
	V6, V7	24 × 24	10.8

The back-side metal layers **MB1** and **MB2** support both PDN routing and signal routing. Using them for BS-PDN decouples power from signal routing, reducing IR drop and freeing front-side metal for timing-critical signals.

EDA Tool Support

The SCCAD-3nm PDK is validated with the following industry-standard EDA tools. Two separate user manuals are provided — one for the Cadence flow (this document) and one for the Synopsys ICC2 flow.

2 Getting Started — PDK Installation from GitHub

2.1 Cloning the Repository

The SCCAD 3nm PDK is hosted on GitHub. Use the following commands to clone the repository.

HTTPS:

```
git clone https://github.com/sjung366/SCCAD-3nm-PDK.git
cd SCCAD-3nm-PDK
```

SSH (recommended for development):

```
git clone git@github.com:SCCAD-Lab/SCCAD-3nm-PDK.git
cd SCCAD-3nm-PDK
```

If the repository uses Git submodules, initialize them as follows:

```
git submodule update --init --recursive
```

2.2 Environment Setup

Before running any EDA tool, configure the following environment variables manually in your shell environment (e.g., **.bashrc** or **.cshrc**).

Required Environment Variables:

```
# Root directory of the PDK repository
export PDK_ROOT=/path/to/sccad-3nm-pdk

# Technology LEF path (Innovus)
export TECH_LEF=$PDK_ROOT/TECH-LEF

# Cell LEF path
export LEF_PATH=$PDK_ROOT/LEF

# Liberty timing library path
export LIB_PATH=$PDK_ROOT/LIB_DB

# QRC tech file path (Quantus)
export QRC_TECH=$PDK_ROOT/QRC

# ITF path
export ITF_PATH=$PDK_ROOT/ITF

# SPICE model path
export MODEL_PATH=$PDK_ROOT/model
```

Verify the key PDK files are accessible:

```
echo $PDK_ROOT
ls $TECH_LEF
```

```
ls $LEF_PATH
ls $LIB_PATH
ls $QRC_TECH
```

Required EDA Tools:

The following Cadence tools must be installed and accessible via your system PATH. License servers should be configured according to your institution's setup.

- **Cadence Innovus** — Primary tool for place-and-route. The **innovus** executable must be accessible from the command line.
- **Cadence Quantus** — Parasitic extraction tool. Used to generate SPEF files from the routed layout using the PDK QRC tech file. The **quantus** executable must be accessible.
- **Cadence Tempus** — Static timing analysis and sign-off tool. The **tempus** executable must be accessible.
- **Cadence Genus** — Logic synthesis tool for generating the gate-level netlist from RTL. The **genus** executable must be accessible. (Required for synthesis; optional if a pre-synthesized netlist is provided.)

System Requirements:

Item	Requirement
Operating System	RHEL 7/8, CentOS 7, or compatible Linux (64-bit)
Shell	bash or csh/tcsh
TCL	Version 8.5 or later
Python	Version 3.8 or later (required for utility scripts)
Disk Space	Minimum 10 GB free for PDK files and run directories
Memory	Minimum 32 GB RAM recommended for full PnR runs

License Configuration: Cadence tools require a valid CDS_LIC_FILE license server:

```
export CDS_LIC_FILE=5280@your.license.server
# Verify tool availability
which innovus
which quantus
which tempus
```

3 Running PnR with Cadence Innovus

3.1 Essential PDK Files for PnR

Before launching a PnR run, it is important to understand which PDK files are loaded at each stage of the Innovus flow. Unlike Synopsys ICC2, which uses the NDM format, Cadence Innovus loads technology LEF, cell LEF, and Liberty files directly. Parasitic extraction uses Quantus with a QRC technology file derived from the ITF. The table below lists all essential files.

Directory / File	Format	Tool	Description
`TECH-LEF/3nm_GAA_FSPR_tech.lef`	.lef	Innovus	Technology LEF: layer definitions, via rules, routing directions
`LEF/3nm_GAA_FSPR.lef`	.lef	Innovus	Cell LEF: macro dimensions and pin geometry
`LIB_DB/3nm_GAA_FSPR_lvt_nldm.lib`	.lib	Innovus/Tempus	Liberty: timing arcs, power tables
`QRC/3nm_GAA_FSPR.tch`	.tch (QRC tech)	Quantus	QRC technology file for parasitic extraction

The **technology LEF** and **cell LEF** define the physical design rules and cell abstracts loaded at Innovus startup. The **Liberty (.lib)** file provides timing and power data for placement optimization and sign-off with Tempus. The **QRC technology file** is the Quantus-native RC model derived from the ITF and is used for full-chip parasitic extraction after routing.

3.2 PnR Flow Overview

The SCCAD-3nm PDK supports a complete RTL-to-GDS back-end flow using Cadence Innovus. The overall flow is illustrated in **Figure 3**, spanning from RTL synthesis through physical implementation and PPA analysis. Each stage is described in detail below.

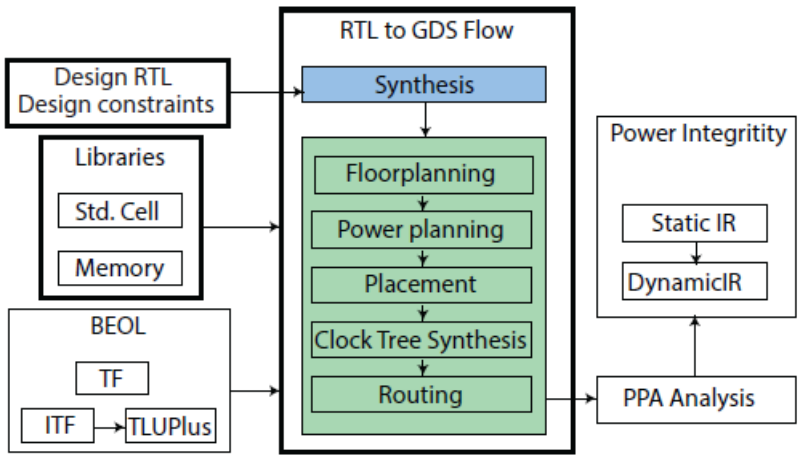

Figure 3. RTL-to-GDS PnR flow with the SCCAD-3nm PDK and Cadence Innovus.

Step 1 — RTL Synthesis

The flow begins with logic synthesis using **Cadence Genus**. The user provides RTL source files and SDC constraints specifying target clock frequency and I/O delays. Genus maps the RTL onto the SCCAD-3nm standard-cell library and produces a gate-level netlist and synthesized SDC, which are passed to Innovus for physical implementation.

```
genus -batch -files scripts/genus/run_syn.tcl \
-log logs/syn.log
```

Step 2 — Design Setup in Innovus

Innovus loads design data using the **read_physical** and **read_timing** commands. Unlike ICC2's NDM, Innovus reads technology LEF, cell LEF, and Liberty files directly at startup. The gate-level netlist and SDC are then loaded to initialize the design database:

```
# Load technology and cell LEF
read_physical -lef [list \
$TECH_LEF/3nm_GAA_FSPR_tech.lef \
$LEF_PATH/3nm_GAA_FSPR.lef]

# Load timing library
read_timing -lib $LIB_PATH/3nm_GAA_FSPR_lvt_nldm.lib

# Load netlist and constraints
read_netlist results/syn/netlist.v
read_sdc results/syn/constraints.sdc
init_design
```

Step 3 — Floorplanning and Power Planning

Floorplanning defines the die area and core utilization. Power planning creates the VDD/VSS power mesh. For BPR-based designs, the buried power rails are pre-defined in the cell LEF; front-side power rings and stripes are added on top:

```
# Initialize floorplan
floorPlan -coreMarginsBy die \
-site FreePDK45_38x28_10R_NP_162NW_34O \
-coreSpacings 2 2 2 2

# Add power domains
add_power_domains -primary VDD -ground VSS

# Create power mesh on upper metal layers
create_pg_mesh_pattern -layers {M4 M5} \
-nets {VDD VSS}
compile_pg
```

Step 4 — Placement

Innovus performs timing-driven placement using **place_design**, followed by post-placement optimization with **optDesign -preCTS**. The tool uses ideal clock assumptions at this stage:

```
place_design
optDesign -preCTS
```

Step 5 — Clock Tree Synthesis (CTS)

Clock Tree Synthesis is performed using **ccopt_design**, Innovus's constraint-driven CTS engine. NDR (Non-Default Routing) rules from the PDK technology file are applied for clock net shielding and spacing. Post-CTS optimization follows immediately:

```
# Create CTS spec from SDC clocks
create_ccopt_clock_tree_spec

# Run CTS
ccopt_design

# Post-CTS optimization
optDesign -postCTS
```

Step 6 — Routing

Global and detail routing are performed by **route_design**, followed by post-route optimization with **optDesign -postRoute**. The router uses the metal stack and spacing rules from the technology LEF. For designs using back-side metal layers (MB1/MB2), additional routing directives may be required:

```
route_design
optDesign -postRoute
```

Step 7 — PPA Analysis

After routing, the final Power, Performance, and Area (PPA) metrics are evaluated. **Performance** is assessed through static timing analysis using **Cadence Tempus**, reporting worst negative slack (WNS) and total negative slack (TNS). **Power** analysis uses switching activity from simulation to report dynamic and leakage power. **Area** is reported directly from Innovus after routing:

```
# Static timing analysis — Tempus
tempus -f scripts/tempus/run_sta.tcl \
-log logs/sta.log

# Area report — Innovus
report_design_area
report_utilization
```

Upon successful sign-off, stream out the final GDS from Innovus:

```
streamOut results/final/design.gds \
-mapFile scripts/gds/layermap.map \
-libName my_design
```

4 PDK Development and Modification

4.1 Overall Flow of PDK Development

The SCCAD-3nm PDK was developed through a structured multi-stage flow integrating TCAD device simulation, physical cell design, parasitic extraction, and library characterization. The full development pipeline is illustrated in **Figure 4**. The yellow-highlighted boxes represent the final PDK deliverables consumed by the PnR flow.

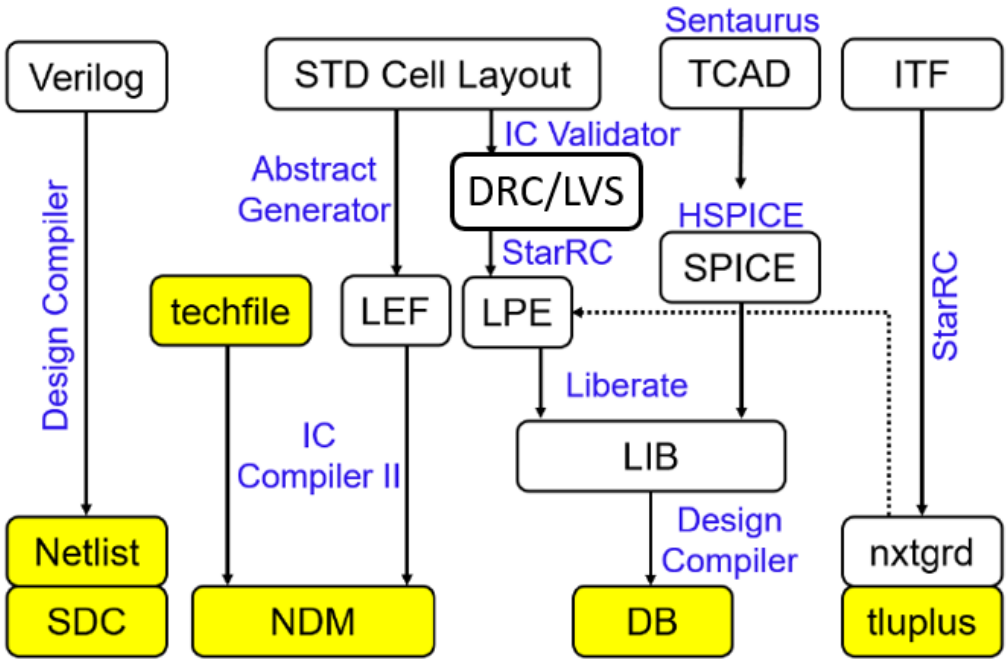

Figure 4. SCCAD-3nm PDK development flow.

Device Modeling — TCAD to SPICE

PDK development begins with **TCAD simulation** using Synopsys Sentaurus to characterize the 3nm GAAFET device. Simulated characteristics are fitted to a BSIM-CMG compact model, producing the **SPICE model card** used in both LPE simulation and Liberty characterization.

Physical Cell Design — Layout to LEF

Standard cell layouts are verified through **DRC/LVS** using Synopsys IC Validator. An **Abstract Generator** extracts the cell abstract view to produce the **LEF** file. The technology file is also used to generate the Cadence-compatible tech LEF for direct loading into Innovus.

Parasitic Extraction — LPE and QRC Tech File

Layout Parasitic Extraction (**LPE**) is performed using **StarRC** driven by the ITF. The ITF is also processed to generate the **QRC technology file** (.tch) for Cadence Quantus. This QRC tech file

encodes the RC model of the BEOL stack in Quantus-native format for full-chip extraction after Innovus routing.

Library Characterization — LPE to LIB

Cell characterization is performed by **Liberate** (Cadence) using HSPICE. Each cell is simulated with LPE parasitics and SPICE model cards across all PVT corners, sweeping input slew and output load. The NLDM data is written out as a **Liberty (.lib)** file for direct use in Innovus and Tempus.

PDK Deliverables

The Cadence PnR flow consumes: **tech LEF + cell LEF** for physical data, **Liberty (.lib)** for timing, and **QRC tech file** for parasitic extraction. Any upstream modification (SPICE model or ITF) requires re-running LPE, QRC generation, and Liberty characterization to maintain PDK consistency.

4.2 PDK Component Modification

4.2.1 SPICE Model

SPICE model cards define the electrical behavior of transistors used in circuit simulation and cell characterization. The SCCAD-3nm PDK provides HSPICE-format model cards for both NMOS and PMOS devices across five PVT corners. Modifying the SPICE model is necessary when target process parameters change — for example, when exploring a different threshold voltage flavor, adjusting device geometry, or fitting the model to new TCAD simulation data.

File location:

```
$PDK_ROOT/model/hspice/
■■■ sccad3nm_tt.l # Typical-Typical (TT) corner
■■■ sccad3nm_ss.l # Slow-Slow (SS) corner
■■■ sccad3nm_ff.l # Fast-Fast (FF) corner
■■■ sccad3nm_fs.l # Fast-Slow (FS) corner
■■■ sccad3nm_sf.l # Slow-Fast (SF) corner
```

Challenge: Manual Parameter Tuning

A BSIM-CMG model card contains tens of interdependent parameters — including threshold voltage (**phig**), carrier mobility (**u0**, **ua**, **etamob**), saturation velocity (**vsat**), and short-channel effect coefficients (**eta0**, **pdibl2**, **dvtp0**), among others. Manually tuning each parameter by trial and error is extremely time-consuming and prone to inconsistency.

Solution: Automated Curve Fitting Script

The PDK includes **auto_curve_fitting.sp**, which uses HSPICE's **opt2** optimization engine to extract all model parameters simultaneously by minimizing the error against reference Id-V curves from TCAD or measurement. The optimizer runs up to 5000 iterations, typically converging within minutes.

Script structure: auto_curve_fitting.sp

Device under test definition:

```
* NMOS LVT, W=20nm, L=12nm, nfin=1
m1 drain gate 0 bulk nmos_lvt w=2e-8 l=1.2e-8 nfin=1
```

Parameter optimization declarations using **opt2(initial, min, max)**:

```
.param
+ phig = opt2(4.5, 4.4, 4.8 ) $ Gate work function (eV)
+ u0 = opt2(0.025145, 0.005, 0.03 ) $ Low-field mobility
+ etamob = opt2(3.0535, 2.7, 3.3 ) $ Mobility degradation exponent
+ vsat = opt2(1.1e+5, 1.0e+5, 1.3e+5) $ Saturation velocity (cm/s)
+ eta0 = opt2(1.3571, 1.0, 3.0 ) $ DIBL coefficient
+ pdibl2 = opt2(6.4836e-3, 2e-3, 1e-2 ) $ DIBL effect parameter
+ dvtp0 = opt2(-1.2334e-2,-0.013, -0.01) $ Vth shift parameter
+ ... $ (16 parameters total)
```

Optimization run and error metric:

```
* Load reference IV data
.data iv merge
+ file=3nm_Ref_iv.dat vds=1 vgs=2 ids=3
.enddata

* Optimize — 5000 iterations, tight convergence
.dc data=iv optimize=opt2 results=compl2 model=optmod2
.model optmod2 opt itropt=5000 relout=1e-6 relin=1e-8 close=10
.measure dc compl2 err2 par(ids) i(m1) minval=1e-11 ignore=1e-12
```

How to use the script:

- Step 1: Prepare **3nm_Ref_iv.dat** with columns [Vds, Vgs, Ids] from TCAD or target spec.
- Step 2: Adjust initial values and search ranges in the **.param** block if needed.
- Step 3: Run: **hspice auto_curve_fitting.sp > fitting.log**
- Step 4: Copy optimized parameter values from the log into the target corner model card (e.g., **sccad3nm_tt.l**).
- Step 5: Verify by re-running simulation without optimization and comparing Id-Vgs/Id-Vds curves.
- Step 6: Re-run Liberty characterization with Liberate and update the **.lib** file before the next Innovus run.

4.2.2 Interconnect Model (ICT)

In the Cadence Quantus parasitic extraction flow, the interconnect technology is described using the **ICT (Interconnect Characterization Technology)** file format — the Cadence-native equivalent of the Synopsys ITF. The ICT file encodes the physical and electrical properties of every BEOL layer, including metal conductor geometry, sheet resistance, via resistance, and inter-layer dielectric constants. Quantus consumes the ICT file directly to generate the **QRC technology file (.tch)**,

which is then used for full-chip RC extraction during the Innovus PnR flow.

File location:

```
$PDK_ROOT/ICT/
■■■ sccad3nm.ict # Main ICT file for Quantus
■■■ sccad3nm.tch # Generated QRC technology file
```

ICT file structure:

The ICT format uses a hierarchical layer definition syntax. Key sections for each metal and via layer include conductor geometry, resistivity, and dielectric stack properties:

```
Layer M1 {
Conductor {
Resistivity = [value] # Sheet resistance (Ω/sq)
Thickness = [value] # Metal thickness (nm)
Width = [value] # Minimum width (nm)
}
Dielectric {
Permittivity = [value] # Relative permittivity (k)
Thickness = [value] # ILD thickness (nm)
}
}

Via VIA1 {
Resistance = [value] # Via resistance (Ω)
Width = [value] # Via width (nm)
Height = [value] # Via height (nm)
}
```

Challenge: Consistent Resistance Scaling

A full 3nm BEOL stack in the ICT file contains conductor and via resistance entries for every layer — M0 through M9, MBPR, MB1–MB2, and all via layers. When a process change requires globally scaling resistance values — for example, modeling increased metal resistivity due to grain boundary scattering at narrow line widths, or evaluating the impact of a ±10% resistance variation across the entire stack — manually editing every **Resistivity** and **Resistance** field is tedious and error-prone, particularly given the structural differences between the ICT format and the ITF format.

Suggested Solution: Automated ICT Scaling Script

The SCCAD-3nm PDK does not currently include a dedicated ICT scaling script, but one can be developed following the same design principles as the provided **auto_itf_scaling.py**. The core approach — using Python regular expressions to locate and scale specific numeric fields while leaving unrelated parameters unchanged — translates directly to the ICT format with minor syntax adjustments.

The key difference is that the ICT file uses named block fields rather than inline key=value pairs. The target fields to scale are:

- **Resistivity** — Metal sheet resistance within each **Conductor { }** block. Analogous to **RHO=** in the ITF.
- **Resistance** — Via resistance within each **Via { }** block. Analogous to **RPV=** in the ITF.

Based on the **auto_itf_scaling.py** implementation as a reference, an ICT scaling script can be structured as follows:

```
# Suggested structure for auto_ict_scaling.py
# (not included in PDK — develop based on auto_itf_scaling.py)

import re, argparse

def scale_ict_resistance(input_file, output_file, scale_factor, targets):
with open(input_file, 'r') as f:
lines = f.readlines()

updated = []
for line in lines:
# Scale metal sheet resistance (Resistivity field)
if ('metal' in targets or 'all' in targets):
line = re.sub(
r'(Resistivity\s*=\s*)([0-9\.eE+-]+)',
lambda m: m.group(1) + f"{float(m.group(2))*scale_factor:.6g}",
line)
# Scale via resistance (Resistance field)
if ('via' in targets or 'all' in targets):
line = re.sub(
r'(Resistance\s*=\s*)([0-9\.eE+-]+)',
lambda m: m.group(1) + f"{float(m.group(2))*scale_factor:.6g}",
line)
updated.append(line)

with open(output_file, 'w') as f:
f.writelines(updated)
```

Suggested workflow after ICT modification:

- Step 1: Develop **auto_ict_scaling.py** using the regex pattern from **auto_itf_scaling.py** as a reference, adapting the field names to match the ICT format (**Resistivity**, **Resistance**).
- Step 2: Run the script with the desired scale factor and target layer type: **python auto_ict_scaling.py -i sccad3nm.ict -o sccad3nm_scaled.ict -s 1.1 -t all**
- Step 3: Verify the output file — confirm that **Resistivity** and **Resistance** values have been scaled while **Thickness**, **Permittivity**, and geometry fields remain unchanged.

- Step 4: Re-generate the QRC technology file from the modified ICT using Quantus: **quantus -cmd scripts/quantus/gen_qrc.cmd** (update the ICT file path in the Quantus run script accordingly).
- Step 5: Re-run the Innovus PnR flow with the new QRC tech file and compare PPA results against the baseline to quantify the impact of the resistance change.